

PC Program — K-Way Merge Using Min Heap

```
#include <stdio.h>

#define K 3

#define MAX 20

#define N 12

int L[K][4] = {
    {10, 30, 50, 70},
    {20, 40, 60, 80},
    {15, 35, 55, 75}    };

typedef struct {
    int value;
    int list;
    int pos;
} Node;

Node heap[MAX];

int size = 0;

int comparisons = 0;

void swap(Node *a, Node *b) {
    Node temp = *a;
    *a = *b;
    *b = temp;    }

void insert(Node x) {
    int i = size++;
    heap[i] = x;
    while (i > 0) {
        int p = (i - 1) / 2;
        comparisons++;
```

```

        if (heap[p].value <= heap[i].value)
            break;
        swap(&heap[p], &heap[i]);
        i = p;
    }
}

Node extractMin() {
    Node result = heap[0];
    heap[0] = heap[--size];
    int i = 0;
    while (2 * i + 1 < size) {
        int left = 2 * i + 1;
        int right = left + 1;
        int smallest = left;
        if (right < size) {
            comparisons++;
            if (heap[right].value < heap[left].value)
                smallest = right;
        }
        comparisons++;
        if (heap[i].value <= heap[smallest].value)
            break;
        swap(&heap[i], &heap[smallest]);
        i = smallest;
    }
    return result;
}

void displayHeap() {
    printf("[");
    for (int i = 0; i < size; i++) {
        printf("%d", heap[i].value);
    }
}

```

```

        if (i < size - 1)
            printf(", ");
    }
    printf("\n");
}

int main() {
    int output[N];
    int count = 0;
    printf("Initial Lists:\n");
    for (int i = 0; i < K; i++) {
        printf("L%d: ", i + 1);
        for (int j = 0; j < 4; j++)
            printf("%d ", L[i][j]);
        printf("\n");
    }
    for (int i = 0; i < K; i++) {
        Node x = {L[i][0], i, 0};
        insert(x);
    }
    printf("\nInitial Heap: ");
    displayHeap();
    printf("\nK-Way Merge Trace:\n");
    while (size > 0) {
        Node x = extractMin();
        output[count++] = x.value;
        printf("Extracted: %d (L%d)\n",
            x.value, x.list + 1);
        int nextPos = x.pos + 1;
        if (nextPos < 4) {
            Node next = {

```

```

        L[x.list][nextPos],
        x.list,
        nextPos };
    insert(next);    }
    printf("Heap: ");
    displayHeap(); }
    printf("\nMerged Output: ");
    for (int i = 0; i < count; i++)
        printf("%d ", output[i]);
    printf("\nTotal elements: %d\n", count);
    printf("Heap key comparisons: %d\n", comparisons);

    return 0;
}

```

```

Initial Lists:
L1: 10 30 50 70
L2: 20 40 60 80
L3: 15 35 55 75

Initial Heap: [10, 20, 15]

K-Way Merge Trace:
Extracted: 10 (L1)
Heap: [15, 20, 30]
Extracted: 15 (L3)
Heap: [20, 30, 35]
Extracted: 20 (L2)
Heap: [30, 35, 40]
Extracted: 30 (L1)
Heap: [35, 40, 50]
Extracted: 35 (L3)
Heap: [40, 50, 55]
Extracted: 40 (L2)
Heap: [50, 55, 60]
Extracted: 50 (L1)
Heap: [55, 60, 70]
Extracted: 55 (L3)
Heap: [60, 70, 75]
Extracted: 60 (L2)
Heap: [70, 75, 80]
Extracted: 70 (L1)
Heap: [75, 80]
Extracted: 75 (L3)
Heap: [80]
Extracted: 80 (L2)
Heap: []

Merged Output: 10 15 20 30 35 40 50 55 60 70 75 80
Total elements: 12
Heap key comparisons: 21

```

Pairwise Merging of Sorted Lists in C

```
#include <stdio.h>

void merge(int A[], int n, int B[], int m, int C[])
{
    int i = 0, j = 0, k = 0;
    while (i < n && j < m)
    {
        if (A[i] <= B[j])
            C[k++] = A[i++];
        else
            C[k++] = B[j++];
    }
    while (i < n)
        C[k++] = A[i++];
    while (j < m)
        C[k++] = B[j++];
}

int main()
{
    int L1[] = {10, 30, 50, 70};
    int L2[] = {20, 40, 60, 80};
    int L3[] = {15, 35, 55, 75};
    int temp[8];
    int result[12];
    int i;
    merge(L1, 4, L2, 4, temp);
```

```
printf("After merging L1 and L2:\n");  
for (i = 0; i < 8; i++)  
    printf("%d ", temp[i]);  
merge(temp, 8, L3, 4, result);  
printf("\n\nFinal merged list:\n");  
for (i = 0; i < 12; i++)  
    printf("%d ", result[i]);  
return 0;  
}
```

```
After merging L1 and L2:  
10 20 30 40 50 60 70 80
```

```
Final merged list:  
10 15 20 30 35 40 50 55 60 70 75 80
```

```
Process returned 0 (0x0)    execution time : 4.922 s
```